

Activity 6: CPU performance on clouds and local environments

The objective of this activity is for you to measure and analyze how cloud instances perform compared to your local machine.

Part 1: Write `cpu_test.py` to measure CPU performance

Write a python program called "`cpu_test.py`" that will loop 50,000 times.

In each iteration, the program simply gets the current time by calling `time.perf_counter_ns()` and appends the timestamp into a **list in memory**.

After completing 50,000 iterations, then have your program write all the saved timestamps from the list in memory to a file on disk. **Do not write to a file on disk inside your loop.** Why not? Because, we want to measure only CPU performance. We do not want any I/O performance to interfere with the CPU measurements.

The file ought to look something like this where the first column is the iteration and the second column is the time (in seconds) **since** the 0'th round.

```
0, 0.00000000
1, 0.00000286
2, 0.00001502
3, 0.00001597
4, 0.00001693
5, 0.00001884
6, 0.00002003
7, 0.00002098
8, 0.00002193
9, 0.00002384
10, 0.00002503
11, 0.00002599
```

Generally, we would expect that if the program is executed continuously, all loop iterations should take a similar amount of time.

Part 2: Running `cpu_test.py` to collect CPU measurements

You will run `cpu_test.py` in 3 scenarios.

1. (Cloud Instance No Credits) Run once in an Amazon EC2 t2.micro instance (free tier) with no CPU credits left. You must completely deplete the CPU credits yourself. This could take 30-40 minutes. So, start early. (Hmm...why does it take so long?)
2. (Cloud Instance with Credits) Run once in an Amazon EC2 t2.micro instance (free tier) that has just been provisioned and has sufficient CPU credits to run your program.
3. (Physical Machine) Run once on your notebook computer. If you are using **Windows**, make sure you are using **Python 3.7+** or your results will not be valid.

Make sure when you create your EC2 instances that you DO NOT select UNLIMITED mode for burstable instances.

While running `cpu_test.py`, take screenshots and make observations on your CPU utilization and CPU credits. Notice the difference in behavior between all 3 scenarios to answer the last question.

Part 3: Comparison

- Create a plot like the one on slide 89 from lecture with **all three scenarios in a single plot.**
- Fill in the following table.

Scenario	1. Cloud t2.micro Instance No Credits	2. Cloud t2.micro Instance with Credits	3. Your Physical Machine
CPU Model and Speed (GHz)			
Total time to run <code>cpu_test.py</code> (seconds)			
Total time to run <code>cpu_test.py</code> (microseconds)			

Part 4: Answer these questions

1. To understand how Clouds manage CPU resources,
 - 1.1. what is a CPU credit in AWS?
 - 1.2. how does it benefit the Cloud user?
 - 1.3. how does it benefit the Cloud provider?
2. From the Cloud user's perspective, which has better (faster) CPU performance: Scenario 1 vs. Scenario 2? Why do you say so? Explain your answer using your plot.
3. Is your notebook faster than a t2.micro/t3.micro instance on the cloud? Explain your answer using your plot.
4. In all 3 scenarios, while `cpu_test.py` is running, is your CPU utilization up to 100%? Explain why/why not for each scenario.

What to submit in mycourseville

- A pdf file with your analysis plot, table, and answers to the questions.
- Your code for `cpu_test.py`

FAQ/Troubleshooting

1. Always check that you are using
 - Amazon Linux 2023 (or Ubuntu)
 - Free tier t2.micro or t3.micro (**Standard Mode**, not Unlimited Mode) instances
 - Check your CPU credit during the experiments (scenario 1, 2 should be different)
 - Python3.7+
2. If your graph does not make sense
 - Check that your code writes to a file **only** after all your loops are done. Why is this important?
 - Check the time function you are using. On some operating systems, you may need to use even higher resolution. For this activity around a microsecond resolution is what we need. Anything coarser is bad. Anything finer is better, but not necessary.
https://docs.python.org/3/library/time.html#time.perf_counter